Sliding window



Reconstruction



dictionary

- star
- stars
- hip

1. data structure for prev. → prev(i) = a pointer to the start of suffix
2. animated run through.

Q: start from end? or from left?

1. build dp + prev table
2. go from the end to the start

online vs offline algorithm

offline: all data in disk

online:

Why care

input: stream, on the go

..100111100 → 

Top search term this month

1
2
3

dynamic programming



"greedy"

Longest substring without repeating characters

```
s = "abcabcbb"
```

3.

```
String findLongestSubNoRepeat(String input) {
}
```

```
String findLongestSubstringNoRepeat(String s) {
    n = len(s);
    for start = 0 → n-1
        charsSeenSoFar = newSet()
        for end = start → n-1
            if s[end] not in set:
                charsSeenSoFar.add(s[end])
            else,
                break, capture (n - start)
                maxLen.
}
```

array.contains(el) → O(N)

set.contains (eg. abcbad) → O(1)

O(1) ← charsSeenSoFar.

a
ab
abc
~~abcb~~

brute-O = $O(n^2)$ → $|s| = 10^6 \to 10^{12}$

↳ O(n) w/ sliding window

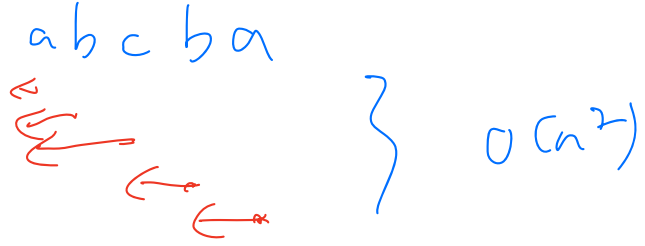
Problem statement

Input: char[] s of length n.

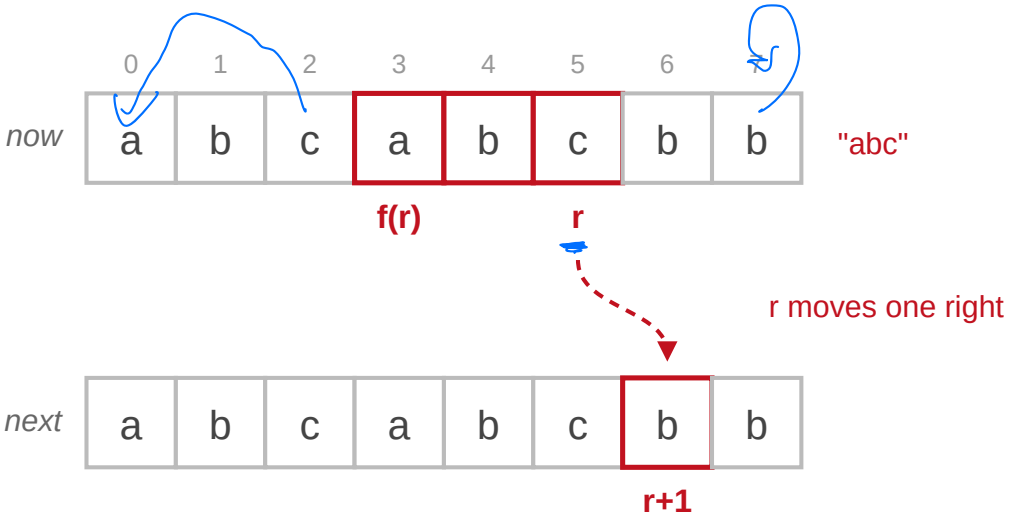
Output: the length of the longest **contiguous** stretch of s in which no value appears twice.

Naive approach

For each r++, walk left until a value repeats.

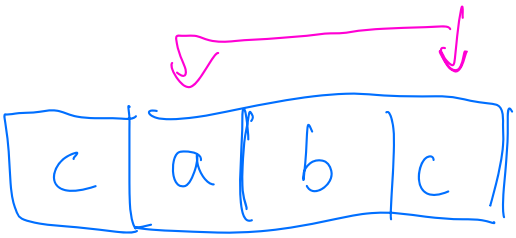


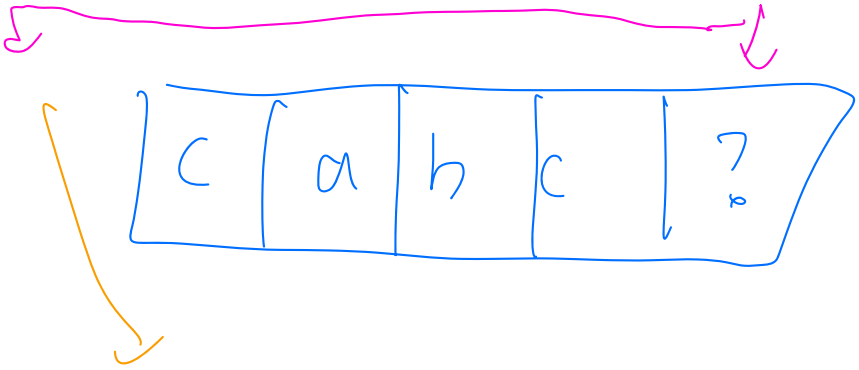
The insight: monotonic f(r)



f(r+1) = ?

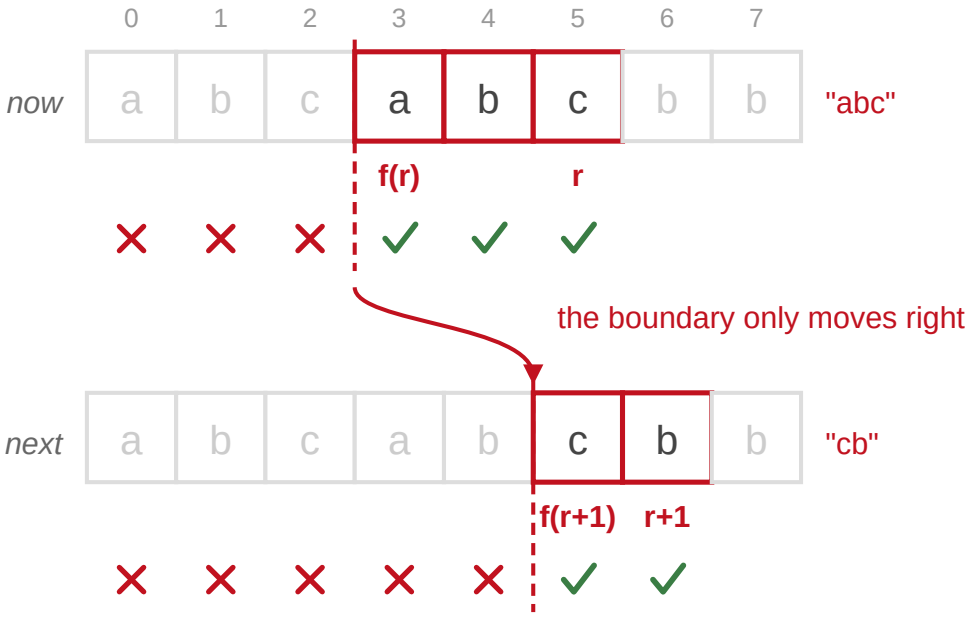
$< f(r)$?



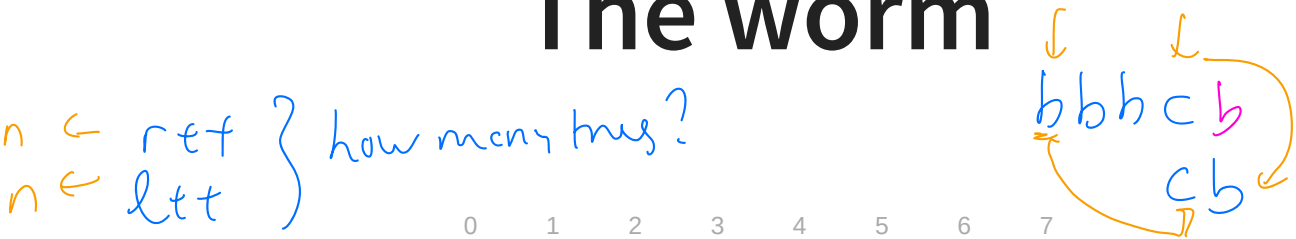


is this possible?

Applicability: Sliding AAABBBB



The worm



n els

↳ Runtime: O(n)

The head moves one step. The tail never moves back.

The sliding window cookbook

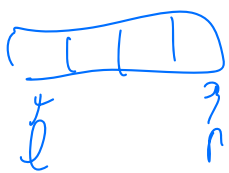
1. **Design** the window property
2. **Prove.** Argue the sliding window property
3. **Code.** Follow the template.

Step 1. Design the window property

Property: I want the longest window that is *good*, where good means ... no duplicate chars in window

Variable: to check that cheaply, I'll track ...

l, r,



→ set ? d b

→ last index ✓

→ map (char: int

b: 3, d: 1

Step 1. Design the window property

Property: all characters unique

Variable: `map[char, int] tally`

Step 2. Argue the sliding window property

(a) For all r, do we have the AAABBB shape?

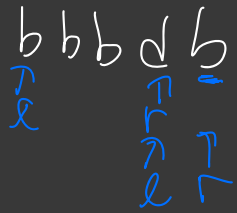
(b) Is f(r) monotonic?

Step 3. Code template

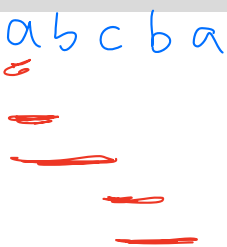
```
l = 0
for r in range(n):
    # 1. add s[r] to the window

    while  # 2. window is bad:
        # 3. remove s[l], l += 1

    # 4. window is good -- update the answer
```

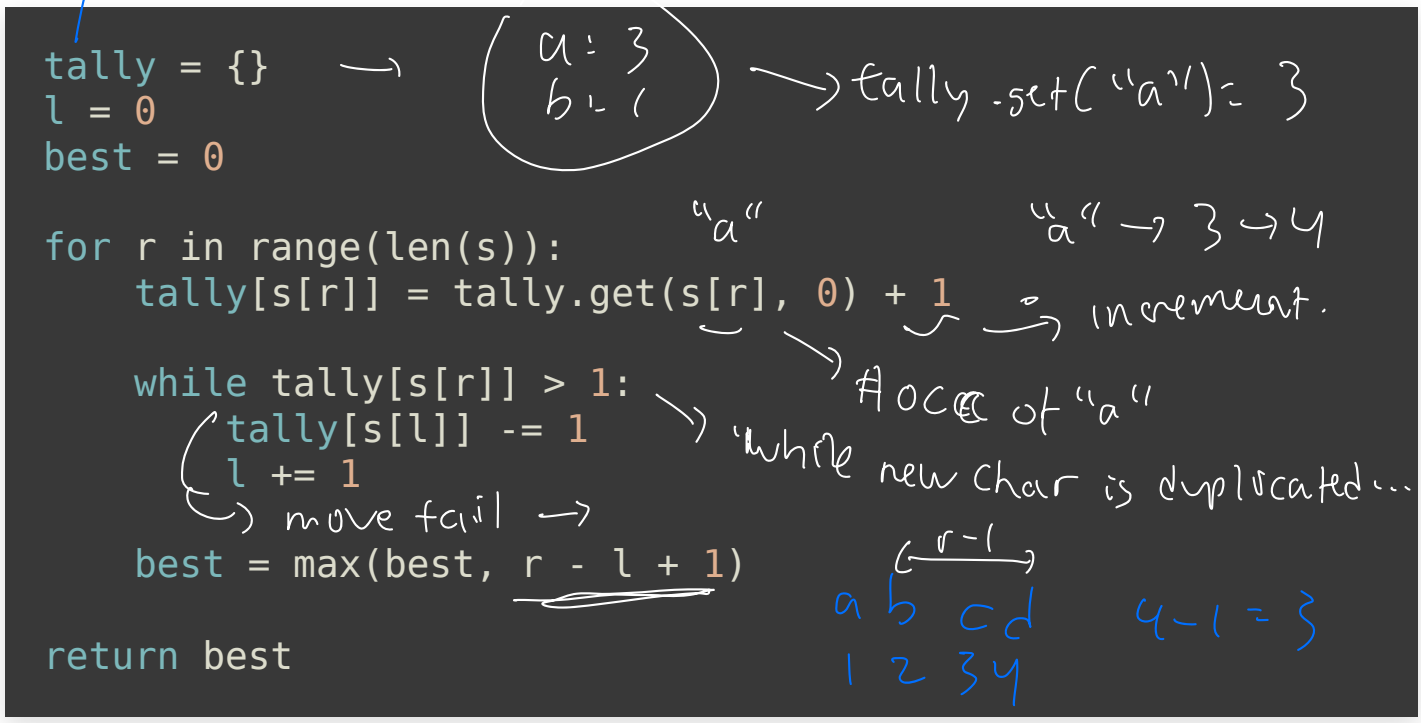






Q: "how many "good" windows are considered by sliding window?

Step 3. Code


k: "a" → letter
v: 3 → #occurrence
tally = {}
l = 0
best = 0
a: 3
b: 1
tally.set("a"): 3
for r in range(len(s)):
tally[s[r]] = tally.get(s[r], 0) + 1
"a"
"a" → 3 → 4
increment.
occ of "a"
while tally[s[r]] > 1:
tally[s[l]] -= 1
l += 1
move fail →
while new char is duplicated...
best = max(best, r - l + 1)
r-l
a b cd
1 2 34
4-1=3
return best

Naive vs sliding window

```
for r in range(n):
    for l in range(r+1):
```

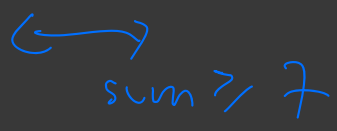
O(n^2) -- l does a full scan every time

```
l = 0
for r in range(n):
    while bad window:
        fix window
        l += 1
```

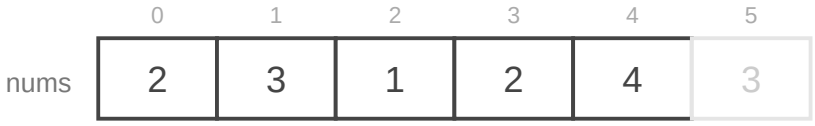
O(n) -- l only goes to the right

Minimum size subarray sum

```
nums = [2, 3, 1, 2, 4, 3]
target = 7
```



Duality



good = sum >= 7



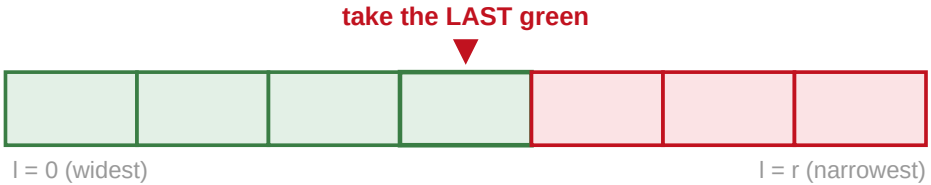
good = sum < 7



Duality

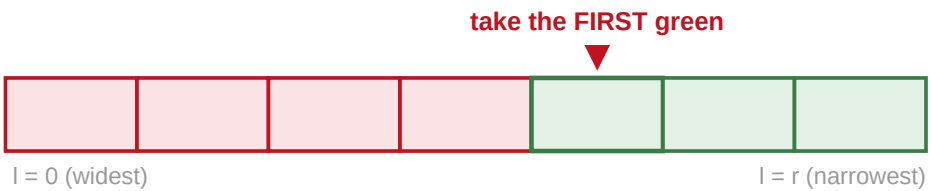
Shortest good window

good = sum >= target. Narrow windows break it.



Longest good window (the dual)

good = sum < target. The colors flip.



Throttler

```
10:00
12:00
10:30
11:00
```

How many requests in the busiest hour?



Problem statement

Input: `log`, a list of times as `"hh:mm"`, in no particular order.

Output: the largest number of entries falling inside any 60-minute window, endpoints included.

↳ find spammest rate.

Steps 1 and 2. Design, prove.

Property: good = `times[r] - times[l] <= 60`

Variable: nothing extra -- the two endpoints are enough.

Monotonicity:

Code

```
times = sorted(to_minutes(t) for t in log)

best = 0
l = 0

for r in range(len(times)):
    while times[r] - times[l] > 60:
        l += 1

    best = max(best, r - l + 1)

return best
```

why no code to put in window?
times[i] captures all states for "sorted" eval

12:00 12:00 12:00 12:00 ... 12:00 13:01

l r

Extension: online



offline: int find_max_hr_rate(int[] timestamps)

timestamps: [11:00, 11:00, 11:30]

online:

Problem setup

```
class ServerHandler:

  # (1) Decide on the class variables

  def ingest(self, user, t):
    # (2) Implement

  def get_max_requests_per_hour(self, user):
    # (2) Implement
```

gets called whenever a user makes a request

→ int

What does the server keep?

What does the server keep?

Per person: a queue of the timestamps inside the current window of 1 hour.

find max # pings in any 1 hour.

Q: "sorry" window?

A. window[r) - win[l) ≤ 60 mins

t = 10:00

Alice	9:15	9:40	9:55	= 3
Bob	9:30			= 1

---

t = 11:00

Alice	~~9:15~~	~~9:40~~	~~9:55~~	10:20	10:45	11:00	= 3
Bob	~~9:30~~	11:00					= 1

Final code

```
def __init__(self):
  # User to queue of times, oldest at the front.
  self.windows = defaultdict(deque)
  self.max_requests = defaultdict(int)

def ingest(self, user, t):
  q = self.windows[user]
  q.append(t)
  while t - q[0] > 60:
    q.popleft()


def get_max_requests_per_hour(self, user):
  # Keep max window len per user and update on ingest
```

k: person name
v: q of timestamps in the last hr

throw away old stuff

Right now:

user 1 → Q1 10:30 11:00
user 2 → Q2 10:00 11:00

insert() → cleans up for that user

More memory optimizations

- global & across all users:
  - bucketize.

Alice 10:00 | Bob 10:01 | ...

- old users stay forever → cron job that cleans old entries every day.

Recap

